



UNIVERSIDADE FEDERAL DA BAHIA - UFBA
RELATÓRIO FINAL DA DISCIPLINA ENGG52 - LABORATÓRIO INTEGRADO I-A

Reconhecimento Facial com CNN em FPGA

Salvador

Junho 2026

Reconhecimento Facial com CNN em FPGA

Trabalho final da disciplina Laboratório Integrado I-A apresentado aos professores Wagner Luiz Alves de Oliveira e Edmar Egídio Purcino de Souza como requisito parcial para obtenção de nota final da disciplina.

Universidade Federal da Bahia - UFBA
Escola Politécnica

Docentes Responsáveis: Prof. Dr. Edmar Egídio Purcino de Souza
Prof. Dr. Wagner Luiz Alves de Oliveira

Salvador
Junho 2026

Lista de Tabelas

Tabela 1 – Métricas de Teste	11
Tabela 2 – Relatório de Classificação do Teste	11
Tabela 3 – Principais saídas do módulo <code>cnn_top</code>	12
Tabela 4 – Principais sinais do submódulo UART	13
Tabela 5 – Principais sinais do submódulo <i>Framebuffer</i> 32×32	13
Tabela 6 – Principais sinais do submódulo <i>Line Buffer</i>	14
Tabela 7 – Principais sinais do submódulo de Convolução	14
Tabela 8 – Principais sinais do submódulo de Camada Densa	15
Tabela 9 – Principais sinais do submódulo Argmax	15
Tabela 10 – Sinais de controle da FSM principal do módulo <code>cnn_top</code>	17
Tabela 11 – Métricas de validação para a Camada de Convolução	19
Tabela 12 – Métricas de validação para as Camadas de <i>Max Pooling</i> e <i>Flatten</i>	19
Tabela 13 – Métricas de validação para a Camada Densa	19
Tabela 14 – Contribuições Individuais	34

Sumário

1	INTRODUÇÃO	6
1.1	Objetivos	6
1.2	Metodologia	7
1.3	Justificativa	7
2	AQUISIÇÃO DE DADOS E TREINAMENTO <i>OFFLINE</i>	8
2.1	Confecção e treinamento da Rede Neural Convolucional (CNN)	8
2.1.1	Classes de 1 a 18: Autorizados (Equipe)	8
2.1.2	Classe 0: Desconhecidos (Unknowns)	8
2.2	Implementação	9
2.3	Inferência e Validação	10
2.3.1	Métricas de Teste e Relatório de Classificação	10
2.4	Quantização e Exportação (.MIF)	11
3	ARQUITETURA DE <i>HARDWARE</i> (RTL)	12
3.1	Gerenciamento de Processamento: Módulo <i>cnn_top</i>	12
3.1.1	Papel e Comportamento dos Submódulos	13
3.1.1.1	UART	13
3.1.1.2	<i>Framebuffer</i> 32×32	13
3.1.1.3	<i>Line Buffer</i>	13
3.1.1.4	Convolução	14
3.1.1.5	<i>Max Pooling</i>	14
3.1.1.6	<i>Flatten</i> (“Achatamento”)	14
3.1.1.7	Camada Densa	15
3.1.1.8	Argmax	15
3.1.1.9	ROM de Pesos	15
3.1.2	Máquina de Estados de Controle	16
3.2	Validação da Arquitetura de <i>Hardware</i>	17
3.2.1	Métricas de Avaliação	18
3.2.2	Resultados das Camadas	19
3.2.3	Análise Gráfica e Conclusões	19
4	SISTEMA DE VISÃO E COMUNICAÇÃO	22
4.1	Protocolo UART	22
4.2	VGA	24
4.2.1	Modo Vídeo	24

4.2.2	Modo Rosto	25
4.2.3	Sprite de nomes	25
4.2.4	Controle de Exibição	25
5	INTEGRAÇÃO FINAL E VALIDAÇÃO	27
5.1	<i>Pipeline</i>	27
5.2	Diagrama de Blocos	27
5.2.1	Fluxo de Dados <i>End-to-End</i>	28
5.3	Validação do Artefato Final	30
5.4	Problemas Encontrados	30
6	CONCLUSÃO	32
7	CONTRIBUIÇÕES INDIVIDUAIS	34
	REFERÊNCIAS BIBLIOGRÁFICAS	36

Resumo

Este trabalho apresenta o desenvolvimento de uma Fechadura Biométrica Inteligente implementada na FPGA DE2-115 (Cyclone IV EP4CE115F29C7) como projeto integrador da disciplina ENGG52 da UFBA. O sistema realiza reconhecimento facial em tempo real através de uma Rede Neural Convolucional embarcada, classificando indivíduos entre 19 classes — 18 membros autorizados e uma classe nativa de rejeição — sem necessidade de limiar de confiança externo. A solução cobre o pipeline completo de ponta a ponta: uma câmera conectada ao PC captura o rosto do usuário, que é detectado pelo algoritmo Haar Cascade, normalizado com CLAHE, redimensionado para 32×32 pixels e transmitido via UART à placa. Na FPGA, uma Máquina de Estados Finitos orquestra o pipeline de inferência composto pelos módulos de convolução 3×3 com ReLU (4 filtros), *Max Pooling* 2×2 , *Flatten* e camada Densa 900×19 , cujos pesos foram treinados em Python com Keras, quantizados para o formato de ponto fixo Q1.7 e armazenados em blocos de memória M9K. O resultado é exibido em um monitor VGA com o nome do indivíduo reconhecido e acionamento de LEDs indicando liberação ou negação de acesso. As validações quantitativas realizadas ao longo do projeto confirmam a fidelidade da implementação em Verilog. O sistema integra os Artefatos 1 a 4 do cronograma proposto, abrangendo aquisição e treinamento off-line, arquitetura RTL, *pipeline* de visão e comunicação, e integração final.

Palavras-chave: FPGA. reconhecimento facial. CNN embarcada. quantização de ponto fixo. verilog. DE2-115.

1 Introdução

O reconhecimento facial tem sido um instrumento amplamente utilizado para contextos de identificação e garantia de segurança. O mapeamento de padrões e características é essencial para o bom funcionamento de um sistema de proteção. Nesse cenário, o desenvolvimento de interfaces homem-máquina (IHM) tem avançado com o aumento da capacidade computacional em sistemas embarcados.

Para traduzir essas características em dados quantizados, as Redes Neurais Convolucionais (CNNs) têm se mostrado altamente eficazes. Em uma aplicação real voltada para a autorização da entrada de um indivíduo cadastrado em um estabelecimento, o fluxo de dados exige a aquisição de imagem em tempo real, o pré-processamento e a inferência no modelo de *Deep Learning*. Contudo, a etapa de inferência impõe um alto custo computacional. Processadores de uso geral e microcontroladores processam as matrizes da rede neural de forma puramente sequencial, o que pode representar um gargalo crítico para a percepção de tempo real do usuário.

Para a implementação da rede neural no FPGA, optou-se pelo desenvolvimento manual dos módulos de *hardware* utilizando a Linguagem de Descrição de *Hardware* (HDL) Verilog. Essa abordagem permite maior controle sobre a arquitetura implementada, possibilitando a exploração explícita do paralelismo inerente às operações convolucionais e de multiplicação-acumulação (MAC) presentes na CNN. Além disso, a implementação manual permite avaliar de maneira mais precisa os benefícios e os custos associados ao desenvolvimento de aceleradores dedicados para aplicações de Inteligência Artificial embarcada.

1.1 Objetivos

Este trabalho tem como objetivo o desenvolvimento de uma *Tiny-CNN*, seu respectivo treinamento utilizando a linguagem de programação Python através de um *dataset* com fotos dos integrantes convertidas em escala cinza, a implementação de seu modelo matemático em linguagem de *hardware* Verilog — além da integração de periféricos, como o protocolo UART e a transmissão de imagem via VGA — e o teste prático do modelo em uma placa FPGA DE2-115.

1.2 Metodologia

Os dados de treinamento foram obtidos através de uma câmera de *notebook*, em sala de aula, em gravações de vídeos de cada integrante, extraíndo os *frames* dos vídeos utilizando a biblioteca openCV em Python e tratando-os em escala cinza e o respectivo treinamento foi realizado mediante Keras. Já o teste de funcionamento do modelo matemático em Verilog baseado na construção do modelo Python foi feito pelo *software* ModelSim, e seu respectivo resultado de simulação comparado com o resultado do modelo Python. O procedimento adotado foi a construção aditiva e gradual da rede em paralelo com implementação e testes de módulos lógicos em Verilog, orquestrados por uma Máquina de Estados Finitos (FSM) confeccionada também paralelamente.

1.3 Justificativa

O trabalho traz contribuições importantes para a literatura ao fazer um estudo comparativo do processo de inferência de redes neurais convolucionais em diferentes arquiteturas. Além disso, é capaz de atuar como referência em possíveis futuras estruturas de fluxos completos de uma CNN em um FPGA.

2 Aquisição de Dados e Treinamento *Offline*

O foco desta etapa foi a criação de um *pipeline* robusto para aquisição de dados, treinamento de uma *Tiny-CNN* otimizada e a exportação dos parâmetros quantizados para o *hardware*. O *dataset* foi projetado para um problema de classificação multiclasse com 19 classes: Classe 0 (Desconhecidos) e Classes 1–18 (Membros autorizados).

2.1 Confeção e treinamento da Rede Neural Convolutacional (CNN)

2.1.1 Classes de 1 a 18: Autorizados (Equipe)

As imagens foram extraídas de vídeos .mp4, utilizando o algoritmo Haar Cascade com scale Factor 1.2 para detecção facial, e houve a implementação do CLAHE (*Contrast Limited Adaptive Histogram Equalization*) para garantir o reconhecimento sob diferentes condições de luz e sombras. Para atingir a meta de 400 imagens por integrante, houve a aplicação de técnicas de aumento de dados incluindo rotação leve (-15° a 15°), variações de zoom (0.9x a 1.1x), espelhamento horizontal e ajustes de brilho.

2.1.2 Classe 0: Desconhecidos (Unknowns)

Para minimizar falsos positivos, a Classe 0 foi expandida para garantir maior diversidade, logo tivemos a integração do LFW (*Labeled Faces in the Wild*) via kagglehub e do UCF *Selfie-dataset*, utilizando de uma proporção de 5:1 (Desconhecidos:Autorizados) para cobrir a variabilidade do rosto humano. Foram aplicadas então as mesmas técnicas de aumento de dados (rotação/zoom) na Classe 0 para garantir a invariância de identidade, além da inclusão de 300 imagens sintéticas (paredes e ruído) para evitar autorizações baseadas em texturas de fundo.

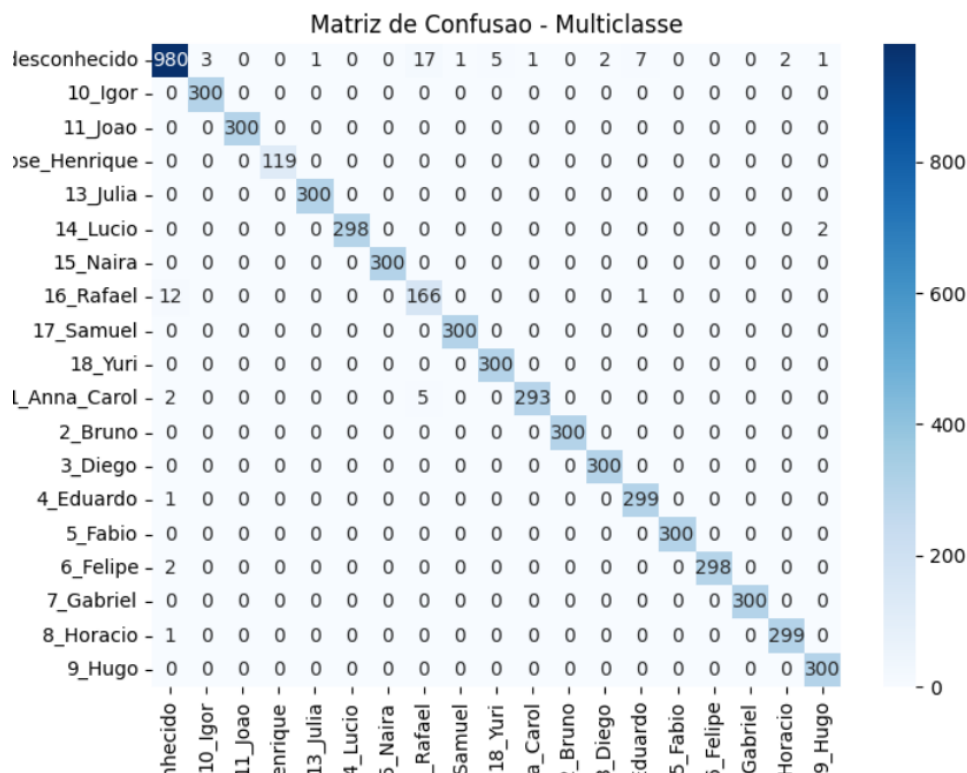


Figura 1 – Matriz de Confusão durante o teste

2.2 Implementação

A rede foi projetada para ser leve (*Tiny-CNN*) visando a implementação em blocos de memória M9K da FPGA, equilibrando capacidade discriminativa e custo computacional compatível com o Cyclone IV EP4CE115F29C7. A arquitetura segue o *pipeline*: Entrada 32×32 (grayscale, Q1.7) $\rightarrow$ Conv2D (4 filtros 3×3 , acumuladores 20 bits) $\rightarrow$ ReLU (combinacional) $\rightarrow$ *Max Pooling* $2 \times 2 \rightarrow$ *Dropout* $\rightarrow$ *Dense* (19 classes) $\rightarrow$ Softmax.

A escolha de apenas 4 filtros convolucionais não é arbitrária — ela resulta diretamente da restrição de *hardware*: cada filtro ocupa um sub-bloco M9K dedicado para armazenar seus 900 pesos densos correspondentes, e o Cyclone IV dispõe de 432 blocos M9K, dos quais 19 são consumidos pelos pesos da camada densa (um por classe) e outros são reservados para framebuffer e line buffer.

A busca de hiperparâmetros foi conduzida com *Keras Tuner (Hyperband)* otimizando simultaneamente a taxa de *Dropout* (0,2 a 0,5), a *Learning Rate* e a regularização L2. A escolha do *Hyperband* sobre o *grid search* convencional foi motivada pela relação classes/parâmetros: com 19 classes e *dataset* balanceado artificialmente, o espaço de hiperparâmetros relevante é estreito e o *Hyperband* converge para ele com menos avaliações completas.

A regularização L2 cumpre papel duplo no projeto: além de reduzir *overfitting*, ela penaliza pesos de grande magnitude durante o treino, o que reduz diretamente o erro de arredondamento introduzido pela quantização posterior para Q1.7 — pesos próximos de zero perdem menos informação relativa ao serem truncados para 8 bits do que pesos de grande amplitude.

O *Custom Loss Weighting* aplicou peso 1,0 à Classe 0 (Desconhecido) e 4,0 às classes de membros autorizados (1–18), refletindo a assimetria de custo do problema: uma falsa negação — negar acesso a um membro autorizado — é operacionalmente mais custosa do que um falso positivo para a classe de rejeição. O treinamento foi conduzido com *EarlyStopping* com paciência de 15 épocas, suficiente para absorver platôs de validação comuns em *datasets* pequenos com *augmentation* agressivo.

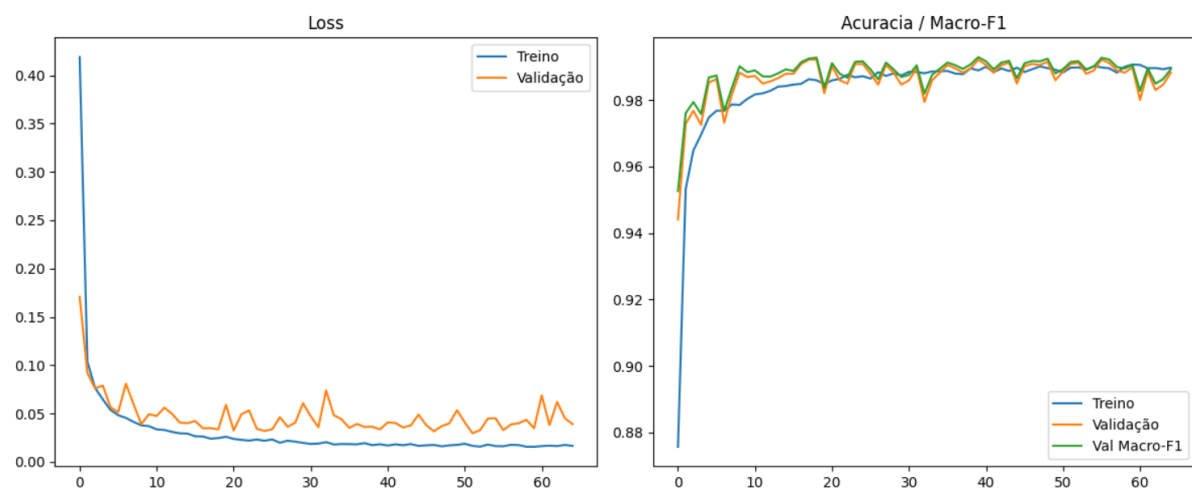


Figura 2 – Matriz de Confusão durante o teste

2.3 Inferência e Validação

O sistema utiliza um script de validação (`inference_webcam.py`) para testagem do desempenho do modelo treinado. Aplica-se o mesmo pré-processamento (CLAHE, Padding de 15%, Redimensionamento) utilizado no treino, e ocorre então a exibição da “Visão da CNN” (32x32) para validação de qualidade com entradas em tempo real. Este fluxo ocorre completamente em *software*.

2.3.1 Métricas de Teste e Relatório de Classificação

Os resultados da avaliação multiclasse durante o teste são dados a seguir:

Tabela 1 – Métricas de Teste

Métrica	Valor
AUC OvR (macro)	0,9999
Macro-F1	0,9901
Balanced Accuracy	0,9918
Recall da classe desconhecido (negar invasor)	0,9608
Recall macro dos alunos autorizados	0,9936

A Tabela 2 detalha as métricas de precisão, revocação (*recall*) e pontuação F1 para cada classe individualmente, bem como as médias globais do sistema.

Tabela 2 – Relatório de Classificação do Teste

Classe	Precision	Recall	F1-score	Support
0_desconhecido	0.98	0.96	0.97	1020
10_Igor	0.99	1.00	1.00	300
11_Joao	1.00	1.00	1.00	300
12_Jose_Henrique	1.00	1.00	1.00	119
13_Julia	1.00	1.00	1.00	300
14_Lucio	1.00	0.99	1.00	300
15_Naira	1.00	1.00	1.00	300
16_Rafael	0.88	0.93	0.90	179
17_Samuel	1.00	1.00	1.00	300
18_Yuri	0.98	1.00	0.99	300
1_Anna_Carol	1.00	0.98	0.99	300
2_Bruno	1.00	1.00	1.00	300
3_Diego	0.99	1.00	1.00	300
4_Eduardo	0.97	1.00	0.99	300
5_Fabio	1.00	1.00	1.00	300
6_Felipe	1.00	0.99	1.00	300
7_Gabriel	1.00	1.00	1.00	300
8_Horacio	0.99	1.00	1.00	300
9_Hugo	0.99	1.00	1.00	300
accuracy			0.99	6118
macro avg	0.99	0.99	0.99	6118
weighted avg	0.99	0.99	0.99	6118

2.4 Quantização e Exportação (.MIF)

Os pesos são convertidos de ponto flutuante para Ponto Fixo (Q1.7) para execução em *hardware* de 8 bits, seguindo a lógica de multiplicação por 128 e arredondamento para o intervalo de -128 a 127. Então, são gerados os ficheiros .mif formatados para os blocos de memória M9K do *Intel Quartus Prime*.

3 Arquitetura de *Hardware* (RTL)

3.1 Gerenciamento de Processamento: Módulo `cnn_top`

O processamento da CNN é gerenciado por um módulo superior (`cnn_top`), que instancia 9 submódulos responsáveis pela inferência da rede, coordenando-os por meio de uma Máquina de Estados Finitos (FSM) própria.

Além dos sinais oriundos dos submódulos instanciados e dos sinais básicos de *clock* e *reset*, o `cnn_top` recebe três sinais externos. Dois deles vêm da lógica do controlador VGA e são utilizados na leitura do *framebuffer*. O terceiro é o *bit* recebido pela comunicação serial, que passa por dois *flip-flops* para mitigar problemas de metaestabilidade antes de ser enviado ao módulo UART.

As principais saídas do `cnn_top` usadas pelo arquivo principal do projeto são detalhadas na Tabela 3.

Tabela 3 – Principais saídas do módulo `cnn_top`

Sinal	Descrição
<code>class_id[4:0]</code>	Valor da classe predita (0 = Vazio, 1 = Desconhecido, 2 a 19 = Autorizados).
<code>access_done</code>	Sinal que indica a finalização do processo de inferência.
<code>frame_ready</code>	Sinal ativo quando o <i>framebuffer</i> 32×32 está completo.
<code>frame_mode</code>	Indica se a exibição será o vídeo completo ou apenas o recorte do rosto.

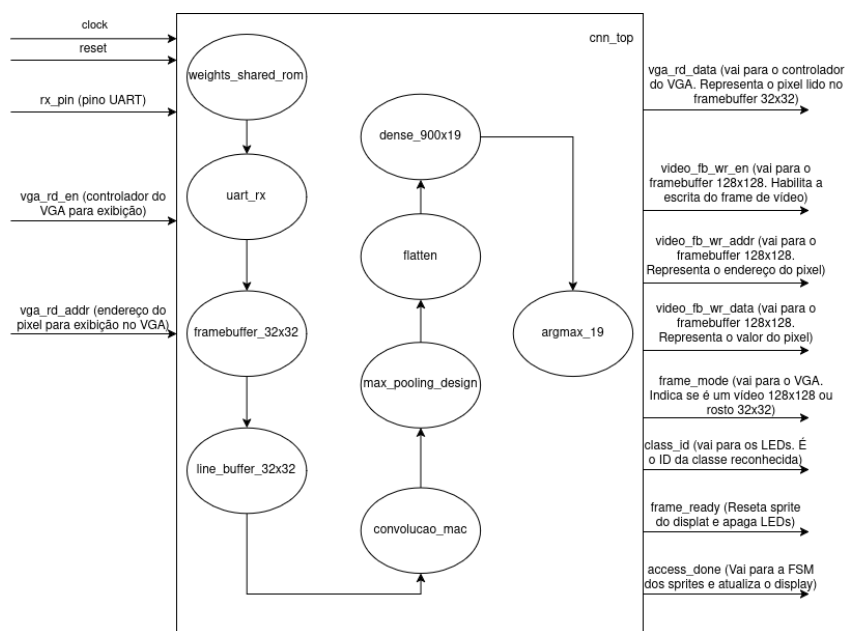


Figura 3 – Diagrama de blocos das principais entradas e saídas do módulo `cnn_top`.

3.1.1 Papel e Comportamento dos Submódulos

O papel e o comportamento de cada submódulo instanciado são detalhados a seguir:

3.1.1.1 UART

Responsável pela decodificação da comunicação serial em *bytes* de 8 *bits*.

Tabela 4 – Principais sinais do submódulo UART

Sinal	Descrição
<code>data_out[7:0]</code>	O <i>byte</i> completo montado após a recepção serial.
<code>data_valid</code>	Pulso indicando que <code>data_out</code> contém um <i>byte</i> válido.

3.1.1.2 *Framebuffer* 32 × 32

Responsável pela escrita espelhada em duas memórias internas distintas de 1024 *bytes*, permitindo a leitura simultânea pela CNN e pelo módulo VGA em frequências diferentes. Durante a escrita, recebe sinais de controle e de endereço do `cnn_top`, processando os dados advindos da UART. Na leitura da rede, a FSM do `cnn_top` envia sinais de requisição e recebe o pixel correspondente. Para a exibição em tela, o controlador VGA faz requisições similares na segunda memória. Ao receber um sinal de *reset*, todos os endereços de ambas as memórias são zerados (0x00).

Tabela 5 – Principais sinais do submódulo *Framebuffer* 32 × 32

Sinal	Descrição
<code>rd_data[7:0]</code>	<i>Byte</i> que representa o pixel solicitado pela CNN durante a leitura do <i>framebuffer</i> .
<code>frame_ready</code>	Pulso que indica quando o último pixel foi armazenado durante a escrita.
<code>vga_rd_data[7:0]</code>	<i>Byte</i> que representa o pixel solicitado pelo VGA durante a leitura do <i>framebuffer</i> .

3.1.1.3 *Line Buffer*

Durante a leitura da imagem, o módulo de *Line Buffer* produz e armazena a janela 3 × 3 a partir do fluxo sequencial de pixels, fornecendo a matriz necessária para a convolução. Como a leitura ocorre da esquerda para a direita e de cima para baixo, o módulo retém o histórico das duas últimas linhas da imagem em blocos de memória dedicados. O deslocamento espacial para formar a janela é coordenado por sinais de controle. Ao associá-las ao pixel atual de entrada, consegue formar a saída desejada.

Tabela 6 – Principais sinais do submódulo *Line Buffer*

Sinal	Descrição
<code>win[0:8][7:0]</code>	A “janela” 3×3 completa formada por cada pixel.
<code>window_valid</code>	Pulso que indica a finalização da construção de uma janela válida.

3.1.1.4 Convolução

Executa a convolução 2D sobre a janela 3×3 gerada pelo *Line Buffer*. Recebe os pesos e vieses dos 4 filtros quantizados em ponto fixo Q1.7. Para cada filtro, a operação de multiplicação e acumulação (MAC) segue a formulação matemática:

$$MAC = \sum_i pixel(i) \cdot peso(i) + bias, \text{ onde } i \in \{0, \dots, 8\}$$

Para evitar *overflow* de informações, os acumuladores são operados em 20 *bits* (quantizados em Q6.14). Em seguida, aplica-se a função de retificação linear (ReLU), na qual valores negativos são descartados (zerados) e valores superiores a 32767 são saturados. O resultado é truncado para 16 *bits* no formato Q2.14. A cada ciclo de *clock*, ele produz 4 saídas (uma por filtro):

Tabela 7 – Principais sinais do submódulo de Convolução

Sinal	Descrição
<code>out_f0</code>	Pixel resultante da convolução da janela com o filtro 0.
<code>out_f1</code>	Pixel resultante da convolução da janela com o filtro 1.
<code>out_f2</code>	Pixel resultante da convolução da janela com o filtro 2.
<code>out_f3</code>	Pixel resultante da convolução da janela com o filtro 3.
<code>valid_out</code>	Pulso que valida a saída da convolução.

3.1.1.5 Max Pooling

Possui quatro memórias para armazenar a linha anterior processada pelos 4 filtros. Por meio de um circuito combinacional, compara os quatro valores de cada sub-janela 2×2 espacial, repassando apenas o valor máximo. O módulo opera com um *stride* de 2, calculando janelas apenas em coordenadas ímpares. Como gera 4 resultados simultâneos e o próximo estágio consome apenas 1 por ciclo, implementa-se um sistema FIFO de 32 posições para serializar as saídas.

3.1.1.6 Flatten (“Achatamento”)

Como a convolução aplicada não possui *padding valid*, a imagem original 32×32 é reduzida para 30×30 . Após o *Pooling*, a dimensionalidade cai para 15×15 por filtro.

Este módulo funciona essencialmente como um contador, serializando cada pixel recebido até atingir os 900 valores esperados ($15 \times 15 \times 4$).

3.1.1.7 Camada Densa

Responsável por realizar a classificação final. O módulo recebe cada pixel proveniente do *Flatten* e realiza o cálculo MAC com 19 pesos simultâneos (um para cada classe), acumulando os resultados ao longo dos ciclos de *clock*. Quando os 900 pixels são finalizados, a camada soma os vieses. Utilizam-se acumuladores de 48 *bits* para reter precisão.

Tabela 8 – Principais sinais do submódulo de Camada Densa

Sinal	Descrição
<code>valid_out</code>	Pulso de controle que informa para o módulo seguinte que o processamento da camada densa foi concluído.
<code>done</code>	Pulso de controle que informa para o <code>cnn_top</code> que o processamento da camada densa se encerrou.
<code>scores[0:18][15:0]</code>	19 valores de 16 <i>bits</i> formatados em Q6.10 que serão usados pelo módulo final para a inferência.

3.1.1.8 Argmax

Módulo que executa o processo final de decisão. Opera como uma FSM que varre as saídas da Camada Densa comparando os *scores*. A classe correspondente ao maior valor é definida como a predição da rede. O índice gerado é internamente incrementado em 1 para se adequar à lógica do VGA, que reserva o ID 0 para a tela vazia (isto é, sem qualquer exibição de nomes).

Tabela 9 – Principais sinais do submódulo Argmax

Sinal	Descrição
<code>valid_out</code>	Pulso de controle para indicar a finalização da inferência.
<code>class_id[4:0]</code>	ID da classe inferida.

3.1.1.9 ROM de Pesos

Módulo responsável por ler o arquivo `.mif` de inicialização e rotear os parâmetros da rede. Ele transfere os pesos para registradores (no caso das camadas iniciais) ou blocos M9K (no caso da camada densa). Para permitir o acesso paralelo aos 19 pesos da camada densa em um único ciclo, estes são distribuídos em 19 blocos M9K individuais. Além dos parâmetros de cada camada, sua saída também inclui um *bit* de controle indicando a conclusão das cópias de cada endereço.

Observações Importantes:

- As saídas dos módulos de *Max Pooling* e *Flatten* são somente os pulsos de controle e o dado processado naquele bloco.
- A camada densa não aplica quaisquer funções de ativação. Isso ocorre por conta da lógica de escolha da saída da rede: a previsão é o ID, isto é, a posição do maior valor calculado. Importante também observar que a classe “Desconhecido” foi tratada como uma classe real, em contraposição à interpretação desta como a negação de todas as demais.

3.1.2 Máquina de Estados de Controle

Conhecendo os principais sinais que o `cnn_top` gerencia, é possível descrever a FSM que controla a CNN conforme o diagrama da Figura 4 e a Tabela 3.

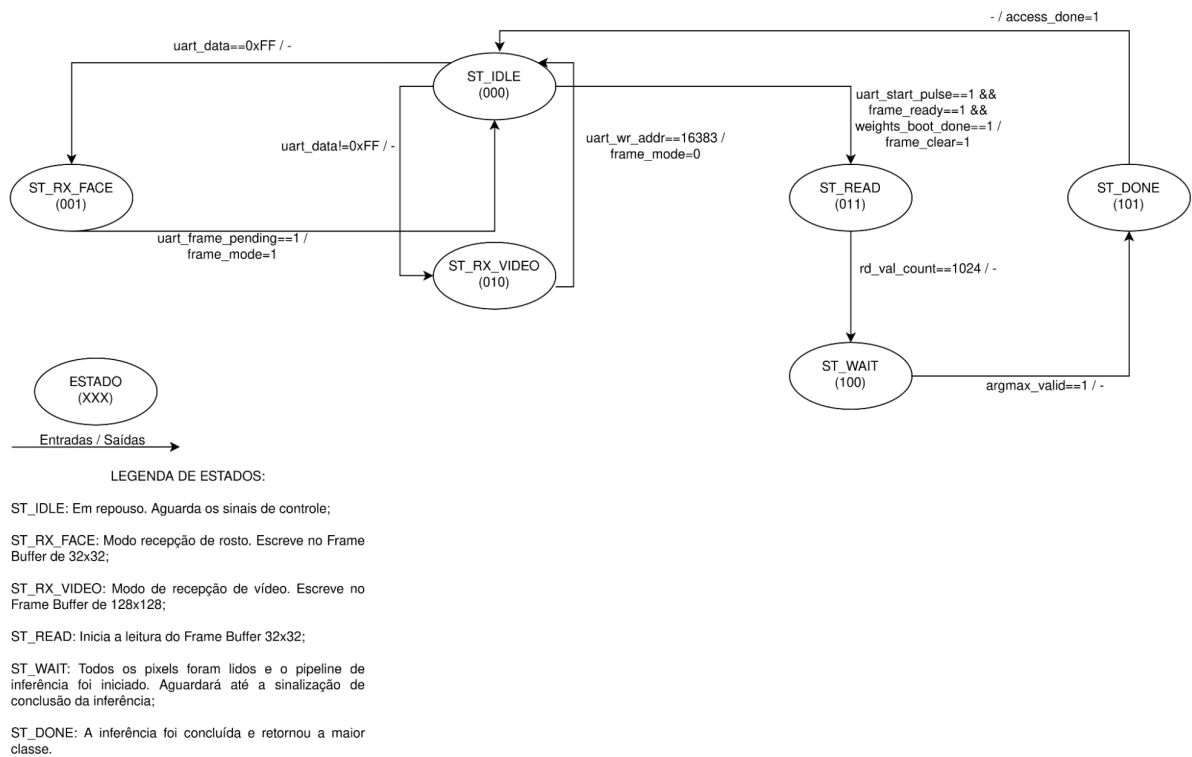


Figura 4 – Diagrama de estados da FSM controladora da CNN no módulo `cnn_top`.

Tabela 10 – Sinais de controle da FSM principal do módulo `cnn_top`

Sinal	Descrição
<code>argmax_valid</code>	Conexão no <code>cnn_top</code> que recebe a saída <code>valid_out</code> do módulo Argmax.
<code>uart_valid</code>	Conexão no <code>cnn_top</code> que recebe a saída <code>data_valid</code> do módulo UART.
<code>uart_data</code>	Conexão no <code>cnn_top</code> que recebe a saída <code>data_out</code> do módulo UART.
<code>uart_wr_addr</code>	Sinal gerado pelo módulo <code>cnn_top</code> durante a leitura do <i>framebuffer</i> . É um contador que incrementa a cada novo <i>byte</i> recebido.
<code>uart_frame_pending</code>	Sinal de controle. Enquanto a imagem está sendo lida, é 0. Sobe para 1 apenas quando <code>uart_wr_addr == 1023</code> .
<code>frame_ready</code>	Conexão no <code>cnn_top</code> que recebe a saída <code>frame_ready</code> do <i>framebuffer</i> .
<code>frame_mode</code>	Pulso de saída que indica ao VGA se a exibição deverá ser de um rosto ou do vídeo.
<code>weights_boot_done</code>	Sinal de controle que indica que o processo de gravação de pesos foi concluído.
<code>frame_clear</code>	Sinal enviado para “desligar” o <code>frame_ready</code> assim que a rede inicia uma nova inferência.
<code>rd_val_count</code>	Contador que indica o progresso de leitura da memória síncrona M9K.

3.2 Validação da Arquitetura de *Hardware*

Para validar as inferências em ponto fixo realizadas pelo *hardware*, fez-se necessária a comparação dessas saídas com seus respectivos valores em ponto flutuante, obtidos via *software*. Para tanto, considerando uma mesma imagem de entrada, adotaram-se procedimentos e métricas específicas de avaliação, estruturados da seguinte forma:

1. **Seleção das Camadas:** Os valores comparados correspondem às saídas dos módulos de Convolução, *Max Pooling*, *Flatten*, Camada Densa e ao resultado final da inferência.
2. **Extração e Quantização em Software:** Cada valor foi extraído do modelo em *software* e quantizado para o formato de ponto fixo equivalente ao do *hardware*. Nas camadas de convolução, *Pooling* e *Flatten*, os valores foram multiplicados por 2^{14} e arredondados. Na camada densa, a saída foi calculada sem a aplicação da função de ativação (*Softmax*) e, em seguida, quantizada no formato Q6.10.
3. **Extração em *Hardware* :** Os valores processados fisicamente pelo circuito foram capturados por meio de um *testbench* específico escrito em Verilog.
4. **Estruturação dos Dados:** Ao final das extrações, geraram-se pares de arquivos `.txt` (um contendo as saídas do *software* e outro as do *hardware*) com a mesma estrutura de organização e quantização para cada camada analisada.

De posse de cinco arquivos de *log* para cada ambiente de execução, as seguintes métricas foram calculadas:

3.2.1 Métricas de Avaliação

Erro Absoluto Médio (MAE): Representa a soma das diferenças, em módulo, calculada ponto a ponto e dividido pelo número total de pontos. Expressa a distância absoluta média entre a saída teórica e a saída física.

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\text{HW}_i - \text{SW}_i| \quad (3.1)$$

Onde N é o número total de amostras da camada, HW é o valor computado no *hardware* e SW é o valor extraído no *software*.

Raiz do Erro Quadrático Médio (RMSE): Avalia a magnitude do erro, penalizando desvios maiores. Caso não ocorram valores excessivamente discrepantes (*outliers* decorrentes de *overflow*), espera-se que o RMSE permaneça próximo ao MAE.

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\text{HW}_i - \text{SW}_i)^2} \quad (3.2)$$

Erro Máximo Absoluto (MaxErr): Serve para identificar o desvio do pior cenário dentro da execução de uma camada.

$$\text{MaxErr} = \max_{i=1}^N |\text{HW}_i - \text{SW}_i| \quad (3.3)$$

Percentual de Valores Exatos (%): Mede a taxa de acerto onde houve paridade perfeita entre os valores.

$$\text{Exatidão} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\text{HW}_i = \text{SW}_i] \times 100, \quad (3.4)$$

Onde $\mathbb{1}[\cdot]$ é a função indicadora, assumindo valor 1 quando a condição é verdadeira e 0 caso contrário.

Coefficiente de Correlação de *Pearson* (r): Mede o grau de correlação linear e a proporcionalidade entre as duas distribuições de saída.

$$r = \frac{\sum_{i=1}^N (\text{SW}_i - \overline{\text{SW}}) (\text{HW}_i - \overline{\text{HW}})}{\sqrt{\sum_{i=1}^N (\text{SW}_i - \overline{\text{SW}})^2 \sum_{i=1}^N (\text{HW}_i - \overline{\text{HW}})^2}} \quad (3.5)$$

Onde a barra indica o valor médio.

Este parâmetro foi eleito como o critério de avaliação mais importante devido à natureza do processo de inferência em classificação. Ainda que os valores absolutos

se distanciem devido às perdas de precisão por quantização e arredondamento, a rede preservará sua predição contanto que mantenha a proporcionalidade entre as ativações. Portanto, quanto mais próximo de 1 estiver o coeficiente (r), mais fiel é a preservação da hierarquia da rede implementada.

3.2.2 Resultados das Camadas

A avaliação conjunta dessas métricas foi realizada processando um lote de testes composto por 20 imagens. Os resultados obtidos são descritos nas tabelas a seguir.

Tabela 11 – Métricas de validação para a Camada de Convolução

Métrica	Média	Mínimo	Máximo
MAE	39.28	33.55	45.10
RMSE	66.30	61.56	72.75
MaxErr	327.35	282.00	390.00
Exato%	46.71%	41.30%	54.40%
<i>Pearson</i>	0.9999	0.9997	0.9999

Tabela 12 – Métricas de validação para as Camadas de *Max Pooling* e *Flatten*

Métrica	Média	Mínimo	Máximo
MAE	52.81	42.38	62.04
RMSE	78.01	68.19	87.03
MaxErr	306.85	270.00	344.00
Exato%	30.59%	23.40%	38.90%
<i>Pearson</i>	0.9999	0.9998	0.9999

Tabela 13 – Métricas de validação para a Camada Densa

Métrica	Média	Mínimo	Máximo
MAE	85.91	47.00	137.89
RMSE	135.07	79.84	204.94
MaxErr	374.75	183.00	559.00
Exato%	33.94%	21.10%	57.90%
<i>Pearson</i>	0.9999	0.9999	1.0000

3.2.3 Análise Gráfica e Conclusões

O foco da validação foi garantir a coesão das inferências físicas comparadas às teóricas, e os resultados apontam que todas as predições físicas foram próximas às do modelo em *software*. Os resultados são satisfatórios, visto que o coeficiente de *Pearson* manteve-se superior a 0,9997 em todas as camadas. Como esperado, visto que a camada

Flatten tem a função exclusiva de reorganizar os dados espacialmente, suas métricas foram iguais às da camada de *Max Pooling*.

Adicionalmente, os valores do MAE e do RMSE apresentaram baixa discrepância entre si, atestando a estabilidade e a ausência de distorções no processamento. Os gráficos a seguir (Figuras 5, 6; 7, 8; 9, 10) ilustram dois exemplos analisados: um pertencente a uma classe autorizada e o outro à classe “Desconhecido”.

Os primeiros gráficos (Figuras 5, 6) consistem em um histograma dos erros absolutos. Devido aos erros intrínsecos ao processo de quantização, a qualidade da implementação é comprovada por uma distribuição estreita, centrada no zero e sem caudas.

Os gráficos (Figuras 7 e 8) detalham as ativações da camada densa para cada classe. À esquerda, em azul (ou vermelho, para a classe “Desconhecido”), estão os valores calculados no *software*. À direita, em laranja (ou vermelho para a classe “Desconhecido”), estão os valores extraídos do *hardware*. O eixo X indica os índices das classes, ao passo que o eixo Y exibe os *scores* no formato Q6.10.

Por fim, os últimos (Figura 9 e 10) apresentam uma representação visual da correlação de *Pearson*. Cada ponto no plano é uma coordenada (SW, HW). A concordância ideal entre esses valores é mapeada pela reta $y = x$, tracejada em preto. Nota-se a proximidade de cada um desses pontos com a reta ideal, atestando a fidelidade da implementação em *hardware*.

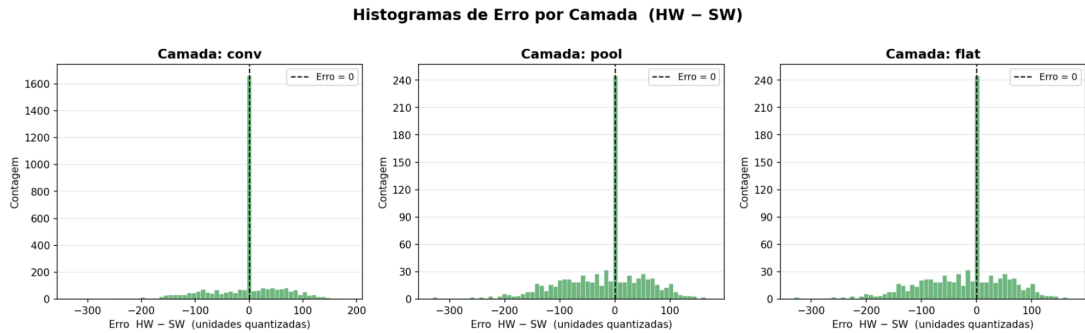


Figura 5 – Histograma de distribuição dos erros absolutos para a classe treinada.

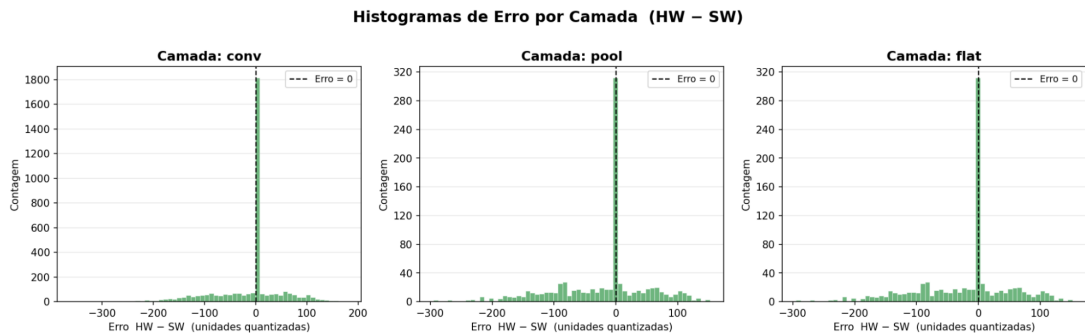


Figura 6 – Histograma de distribuição dos erros absolutos para a classe “Desconhecido”.

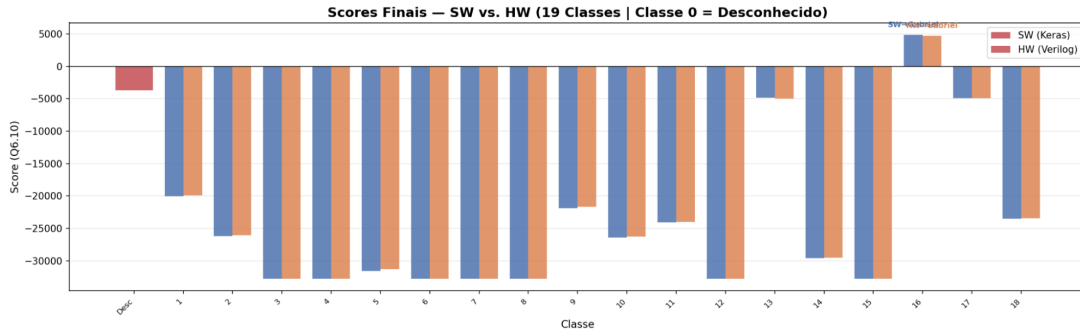


Figura 7 – Comparação das ativações da camada densa para a classe treinada.

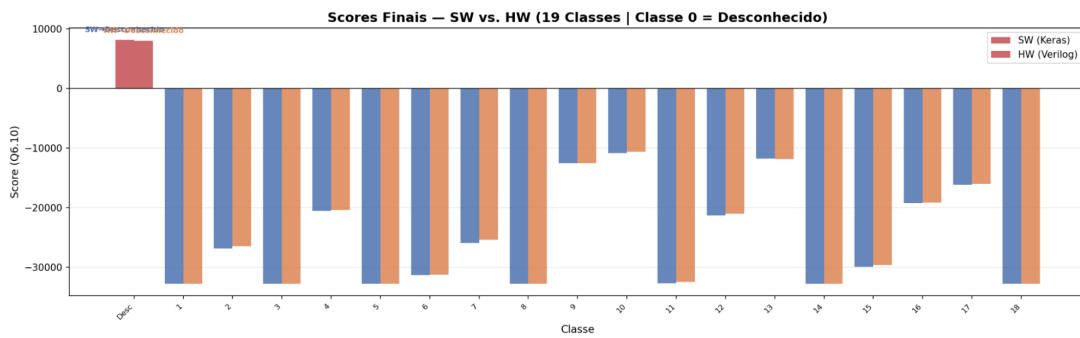


Figura 8 – Comparação das ativações da camada densa para a classe “Desconhecido”.

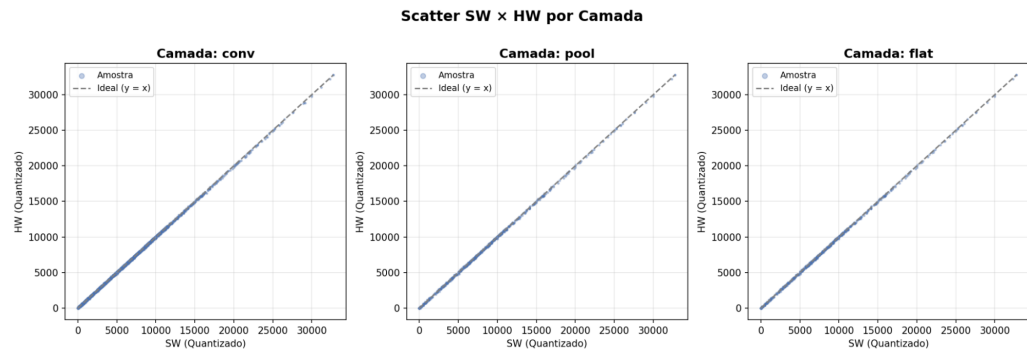


Figura 9 – Dispersão das ativações para a classe treinada.

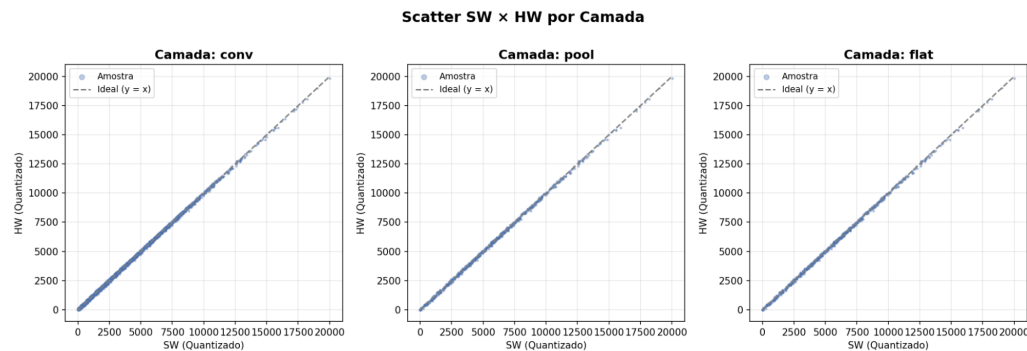


Figura 10 – Dispersão das ativações para a classe “Desconhecido”.

4 Sistema de Visão e Comunicação

4.1 Protocolo UART

A UART atua como o único canal de alimentação de dados dinâmicos para a FPGA. Sem este módulo, a rede neural carece de dados de entrada, impossibilitando o reconhecimento facial. O projeto implementa exclusivamente a via de recepção (RX), resolvendo o problema crítico de transferir dados do mundo de *software* (PC/Python) para o domínio de *hardware* (FPGA) sem um barramento complexo (como PCIe ou USB nativo). A robustez desse módulo é crítica, pois qualquer dessincronização ou erro de enquadramento corrompe a imagem. Os parâmetros e interface física do módulo são os seguintes:

1. Frequência de Operação: 50 MHz (*Clock* nativo da DE2-115).
2. Parâmetros de Comunicação (Padrão): *Baud Rate* configurado e integrado a 2.000.000 bps (2Mbaud), 8 *bits* de dados, sem paridade, 1 Stop Bit.
3. Interface Física: Recepção de sinal TTL 3.3V proveniente de adaptador USB-TTL externo conectado aos pinos de expansão GPIO (JP5).

Como o protocolo é assíncrono, a FPGA não compartilha o relógio com o PC. O sincronismo é calculado pela razão entre a frequência da placa e a taxa de transmissão desejada:

O número de ciclos de *clock* por *bit* é calculado pela razão entre a frequência do *clock* e a taxa de transmissão (*baud rate*):

$$\text{Ciclos por bit} = \frac{f_{\text{clk}}}{\text{Baud Rate}}$$

Para um *clock* de 50 MHz e uma taxa de transmissão de 2 Mbps, obtém-se 25 ciclos de *clock* por *bit*. O submódulo `uart_rx` realiza *oversampling* assíncrono gerenciado por uma Máquina de Estados Finitos (FSM) de quatro estados: IDLE, START, DATA e STOP.

1. IDLE: Em repouso, a linha serial permanece em nível alto. A detecção do *start bit* ocorre por uma borda de descida no pino `rx_pin`.
2. START: A FSM aguarda exatamente metade de um período de *bit* (12 ciclos). Se o pino retornar a '1', a máquina aborta a recepção, descartando o ruído transiente (*glitch*). Se continuar '0', o *start bit* é confirmado.
3. DATA: Os 8 *bits* de dados são amostrados no centro de cada período de *bit* subsequente, sendo reconstruídos por um registrador de deslocamento *LSB-first*.

4. STOP: Ao término da amostragem, o sistema aguarda o *stop bit*. O *byte* completo é disponibilizado na saída `data_out`, acompanhado de um pulso de um ciclo no sinal `data_valid` (*flag de data ready*), retornando ao estado IDLE.

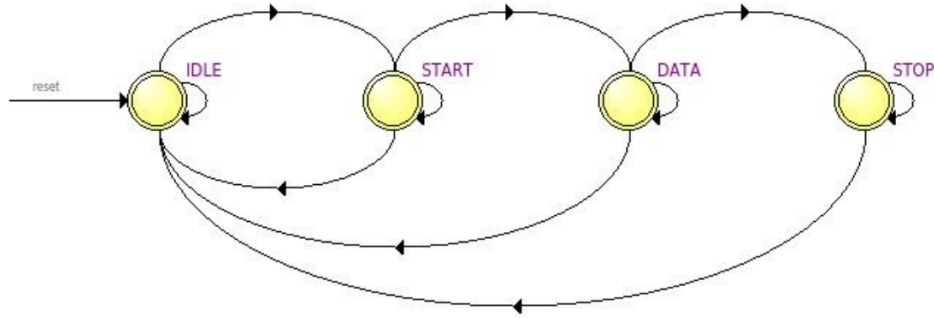


Figura 11 – FSM do UART RX

Devido à assincronia com o mundo externo, o sinal RX passa por um circuito *Double-Sync Flip-Flop* (`rx_sync_1` e `rx_sync_2`) antes de entrar na UART, mitigando a ocorrência de metaestabilidade.

Dentro da arquitetura global, a UART não escreve diretamente na memória. Ela age como um produtor em um padrão produtor-consumidor, alimentando a máquina de estados orquestradora localizada no módulo `cnn_top`. Essa FSM consome os bytes da UART e decide dinamicamente o roteamento dos pixels com base em um protocolo de *Byte de Controle*, demultiplexando o fluxo para exibição (VGA) ou para inferência (CNN). O mapa hierárquico do Caminho UART se dá da seguinte forma:

```

fpga_top_unified.v (Top-Level)
|
|-- cnn_top.v (Orquestrador)
|   |-- Registradores de Double-Sync (rx_sync_1, rx_sync_2)
|   \-- Lógica Combinacional de Demultiplexação (uart_wr_en_comb,
|       video_fb_wr_en)
|
|-- uart_rx.v (Receptor Serial)
|
|-- framebuffer_32x32.v (Consumidor A: Inferência + VGA)
|
\-- framebuffer_128x128.v (Consumidor B: Apenas VGA)

```

O primeiro *byte* recebido de cada novo *frame* atua como uma chave de controle:

Se o *Byte* = 0xFF

O fluxo é roteado para o Modo Rosto. O sinal combinacional `uart_wr_en_comb` é ativado, escrevendo os próximos 1024 bytes na porta A do `framebuffer_32x32`.

Se o *Byte* $\neq$ 0xFF

O fluxo é roteado para o Modo Vídeo. A FSM ativa o sinal `video_fb_wr_en`, gravando os próximos 16384 bytes diretamente no `framebuffer_128x128`. Desta forma, o módulo `uart_rx` atua de maneira dissociada da memória, garantindo que o *pipeline* da rede neural só seja engatilhado mediante o recebimento empacotado e validado de uma matriz pertinente à inferência.

4.2 VGA

Para realizar a transmissão de vídeo entre o FPGA e o monitor VGA, que por padrão tem 640x480 de área, foram estabelecidos valores de offsets de altura (`V_OFFSET` = $(480 - 384) / 2 = 48$) e de largura (`H_OFFSET` = $(640 - 384) / 2 = 128$). Estes valores são subtraídos dos contadores de varredura para a geração das coordenadas locais (`local_x`, `local_y`). Gerando um quadrado de exibição centralizado de 384x384 para as imagens nos modos vídeo e rosto.

Para operar de acordo com o que ocorre nas etapas de validação do *frame* facial ou exibição do vídeo, o VGA depende de uma série de sinais vindos do módulo `cnn_top`. Dentre esses sinais, o sinal `class_id` contém o identificador da classe reconhecida, sendo utilizado para determinar se o *frame* corresponde a uma pessoa cadastrada. O `access_done` informa que o processamento do *frame* foi concluído (*pipeline* terminado) e que o resultado da validação encontra-se disponível para os demais sinais. O sinal `frame_ready` é utilizado para indicar que um novo *frame* foi capturado, sendo usado para controle de LEDs e definição do variável que limpa o *sprite* (exibição de campo sem nome). Por último, o sinal de `frame_mode` chaveia o modo de exibição do VGA.

4.2.1 Modo Vídeo

Neste modo, o sistema precisa exibir o *feed* da câmera, que possui resolução de 128x128 pixels, como usamos uma janela de 384x384, isso exige um fator de ampliação de 3x no vídeo recebido. Para evitar a síntese de divisores em *hardware*, a interpolação ocorre via aritmética de ponto fixo. A coordenada sofre multiplicação por uma constante igual a 2 elevado a 16 dividido por 3, resultando em 21845. O *hardware* então executa um deslocamento de *bits* (operação de *shift*) para o descarte da fração (`vid_x_full[22:16]`).

O endereçamento do framebuffer de 128x128 ocorre por concatenação de eixos (`vid_y`, `vid_x`).

4.2.2 Modo Rosto

Quando o *Haar Cascade* detectar uma face, o sistema foca em uma miniatura de 32x32 pixels, novamente, por termos uma janela de exibição de 384x384, isto exige fator de zoom. A coordenada `local_x` sofre multiplicação por K igual a 2 elevado a 16 dividido por 12, resultando em 5462. Com a extração de 5 *bits* (`fb_x_full[20:16]`), obtém-se a divisão por 12. A vetorização de endereço para o framebuffer de 32x32 ocorre por operação de concatenação (`fb_y`, `fb_x`). A inclusão de nomes ocorre por leitura da `rom_sprites` através da verificação do sinal `display_mode`. Com o sinal em zero, o sistema inibe a leitura de texto. Com o sinal em um, ocorre a comparação dos contadores de pixel com as margens da área de *sprite*. O cruzamento na área (`in_sprite_window`) gera o endereço e a leitura da ROM. O *bit* lido atua como seletor no multiplexador, causando a substituição da cor de imagem pela cor de texto. O retorno do sinal `frame_mode` para zero aciona a limpeza do registrador de identificação (`display_class_id`) e cessa a sobreposição de texto na tela.

4.2.3 Sprite de nomes

Cada *sprite* foi fixado na resolução de 256x32 pixels, ocupando 8.192 endereços físicos na memória. A ROM foi projetada para comportar 19 classes, gerando um total de 155.648 endereços de bit. Para garantir precisão, foi utilizado HTML e CSS para criar a imagem e um script Python feito com uso da biblioteca OpenCV. Este último foi responsável por aplicar uma binarização, definindo os caracteres como nível lógico 0 e 1, e compilar o arquivo de inicialização de memória (`.mif`, referenciado no módulo `rom_sprite`). O tamanho dos blocos (total de *bit* de cada nome), possibilitou o uso de *bit shift* (deslocamento binário) para paginação da ROM ao invés do uso de blocos multiplicadores (DSP).

4.2.4 Controle de Exibição

Para fazer o controle de exibição dos modos foi usada uma arquitetura de chaveamento de *datapath*. O sinal da CNN (`frame_mode`) opera no domínio de *clock* de 50 MHz. Para o cruzamento deste sinal para a interface (25 MHz), ocorre a utilização de dois registradores em sequência (`frame_mode_sync` e logo depois `display_mode`), resultando na geração do sinal `display_mode`. Este sinal governa o Multiplexador 2-para-1, que seleciona o barramento de memória (128x128 ou 32x32) com base na lógica condicional do `display_mode`. Para a compensação de latência de 1 ciclo de *clock* na leitura das memórias,

os sinais de sincronismo (*hsync*, *vsync* e **video_on**) dos geradores de pulso passam por um estágio de retardo.

5 Integração Final e Validação

5.1 Pipeline

O sistema implementa uma fechadura biométrica inteligente cujo fluxo de operação se divide em dois domínios distintos: o domínio de *software*, executado em um computador, e o domínio de *hardware*, sintetizado na FPGA DE2-115. A separação entre os domínios é estabelecida pela interface serial UART a 2.000.000 bps (2Mbaud), que transporta os dados de imagem do PC para a placa de forma unidirecional.

No domínio de *software*, o script Python captura um *frame* da webcam, aplica o algoritmo Haar Cascade para detecção e recorte do rosto com *padding* de 15%, normaliza a iluminação via CLAHE, redimensiona a imagem para 32×32 pixels e quantiza os valores de pixel para o formato Q1.7, resultando em 1024 bytes que são transmitidos serialmente à FPGA. A inferência é disparada automaticamente quando o Haar Cascade detecta um rosto dentro da tolerância de tamanho definida, evitando o envio de *frames* sem rosto e reduzindo classificações incorretas.

No domínio de *hardware*, a FPGA recebe os bytes via módulo `uart_rx`, armazena a imagem completa no *framebuffer* interno de 1024 posições e, ao confirmar o recebimento do último pixel, dispara automaticamente a Máquina de Estados (FSM) de inferência. O resultado — o índice da classe predita — é guardado (*latched*) nos LEDs e no controlador VGA, que exibe o nome do indivíduo reconhecido e o *status* de acesso no monitor.

5.2 Diagrama de Blocos

O sistema é organizado em cinco camadas funcionais hierárquicas, descritas a seguir de cima para baixo:

Camada de Captura (PC): *Webcam* USB → detecção Haar Cascade → pré-processamento CLAHE → quantização Q1.7 → serialização UART.

Camada de Recepção (FPGA): O módulo `uart_rx` decodifica o protocolo 8N1 a 2Mbaud e entrega bytes válidos ao *framebuffer*. O `framebuffer_32x32` mantém duas RAMs espelhadas de 1024×8 bits — uma dedicada ao *pipeline* da CNN e outra ao controlador VGA —, permitindo leituras simultâneas independentes.

Camada de Inferência (FPGA): A FSM `cnn_top` orquestra sequencialmente os módulos computacionais. O `line_buffer_32x32` mantém as duas últimas linhas da imagem para extração de janelas 3×3 em tempo real. O `convolucao_mac` aplica

quatro filtros paralelos com acumuladores de 20 *bits* e ReLU combinacional. O `max_pooling_design` reduz o mapa de 30×30 para 15×15 por canal usando uma FIFO interna de 32 posições para suavização do fluxo. O `flatten` serializa os 900 elementos. A `dense_900x19` mantém 19 acumuladores paralelos de 48 *bits* em formato Q3.21, convertendo para Q6.10 ao final. O `argmax_19` seleciona a classe de maior *score* por comparação em cascata combinacional.

Camada de Pesos (FPGA): O módulo `weights_shared_rom` carrega os 17.159 bytes de parâmetros — 36 pesos convolucionais, 4 *biases* de convolução, 17.100 pesos densos e 19 *biases* densos — a partir do arquivo `weights_all.mif` inicializado nos blocos M9K durante a síntese. Um *bootloader* de *hardware* distribui os pesos da ROM mestre para 19 sub-blocos M9K paralelos, um por classe, permitindo leitura simultânea de todos os pesos densos em um único ciclo de *clock*.

Camada de Exibição (FPGA): O módulo `top_vga` gera os sinais de sincronismo HSYNC e VSYNC para o padrão 640×480 a 60 Hz utilizando uma PLL que deriva 25,175 MHz a partir do *clock* de 50 MHz da placa. A lógica de composição de tela divide a área visível em duas regiões: o rosto capturado lido diretamente do *framebuffer* e o nome do indivíduo reconhecido lido de uma entre 19 ROMs de nome, selecionada pelo `class_id`. Os LEDs verdes acendem em padrão binário com o `class_id` predito, sinalizando a inferência concluída ou indicando a classe “Desconhecido”.

5.2.1 Fluxo de Dados *End-to-End*

O projeto é composto por dois *pipelines* que compartilham o mesmo *framebuffer* de imagem:

- **Pipeline CNN (50 MHz):** Recebe a imagem, processa as camadas da rede neural (convolução, *max pooling*, camada densa) e identifica a classe.
- **Pipeline VGA (25 MHz):** Lê a mesma imagem do *framebuffer* e a renderiza no monitor, junto com um *sprite* de texto contendo o nome da classe predita.

O script Python captura um *frame* da webcam e tenta detectar um rosto via Haar Cascade. Se um rosto é detectado, o script envia o byte de controle `0xFF` seguido de 1024 bytes (imagem 32×32 em escala de cinza, quantizada em Q1.7). Se não detectar um rosto, envia `0x00` seguido de 16.384 bytes (imagem 128×128 para exibição apenas). A FPGA, então, armazena a imagem no *framebuffer* correspondente. O controlador VGA lê continuamente e exibe a imagem ampliada ($12\times$ para rosto, $3\times$ para vídeo) centrada no monitor (384×384 pixels na região central de 640×480).

Para *frames* de rosto, o *pipeline* da CNN processa automaticamente: convolução com 4 filtros 3×3 , *max pooling* 2×2 , camada densa $900 \rightarrow 19$ e *argmax*. O *class_id* resultante determina qual *sprite* de nome aparece na parte inferior da tela (verde para pessoa reconhecida, vermelho para desconhecido) e qual LED acende.

Diante disso, o módulo de mais alto nível da implementação (*fpga_top_unified*) é responsável por integrar o fluxo da CNN com a exibição no VGA. Simplificadamente, ele pode ser representado pelo diagrama da Figura 12.

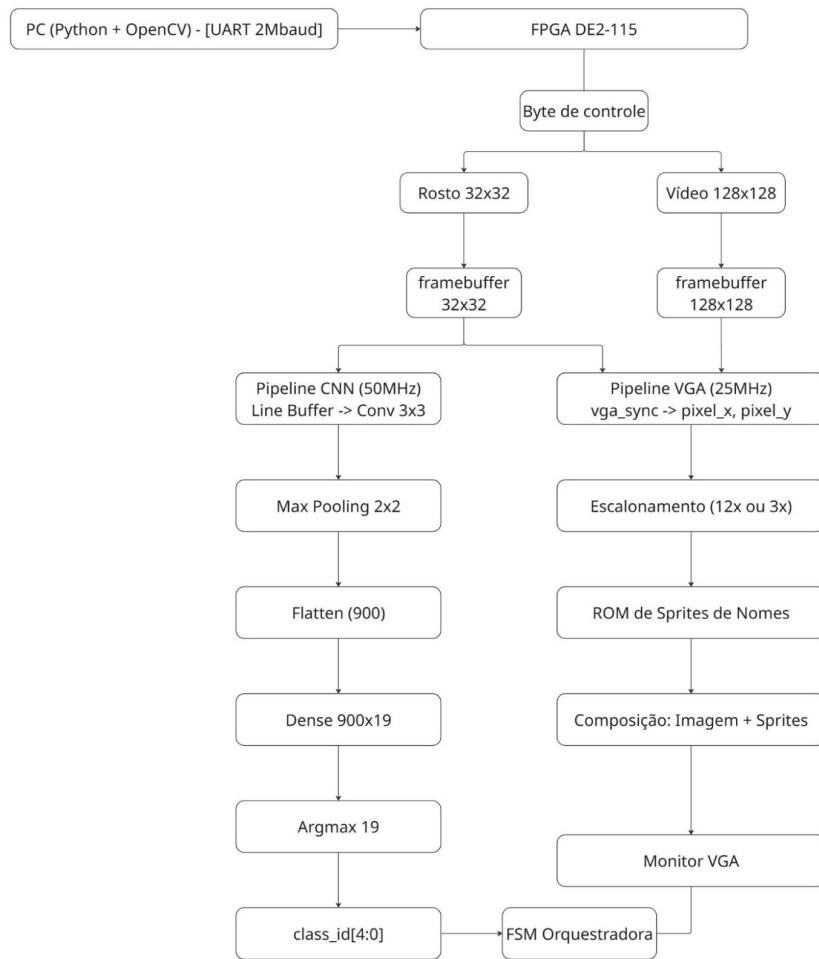


Figura 12 – Diagrama de blocos do módulo *fpga_top_unified*.

Assim, suas funções incluem: gerenciamento do *clock* do VGA através do *vga_pll*, mapeamento das coordenadas para exibição em tela e renderização da imagem final. Além disso, é também responsável por comandar o fluxo entre a CNN e o VGA. Diferentemente dos demais módulos de controle, com estados e caracterizações explícitas de FSM, o *fpga_top_unified* coordena essas duas implementações por meio de um registrador

(o `display_class_id`). Em resumo, seu valor é mantido sempre em 0 até receber a confirmação de que a inferência foi concluída. Neste instante, seu valor passa a ser o mesmo do sinal de saída da CNN (o `class_id`, descrito acima).

Esse comportamento é o que explica a existência do “vazio” dentro dos *sprites*. Para limpar a exibição dos nomes, o registrador simplesmente transita entre um nome válido e o 0, conforme o diagrama da Figura 13.

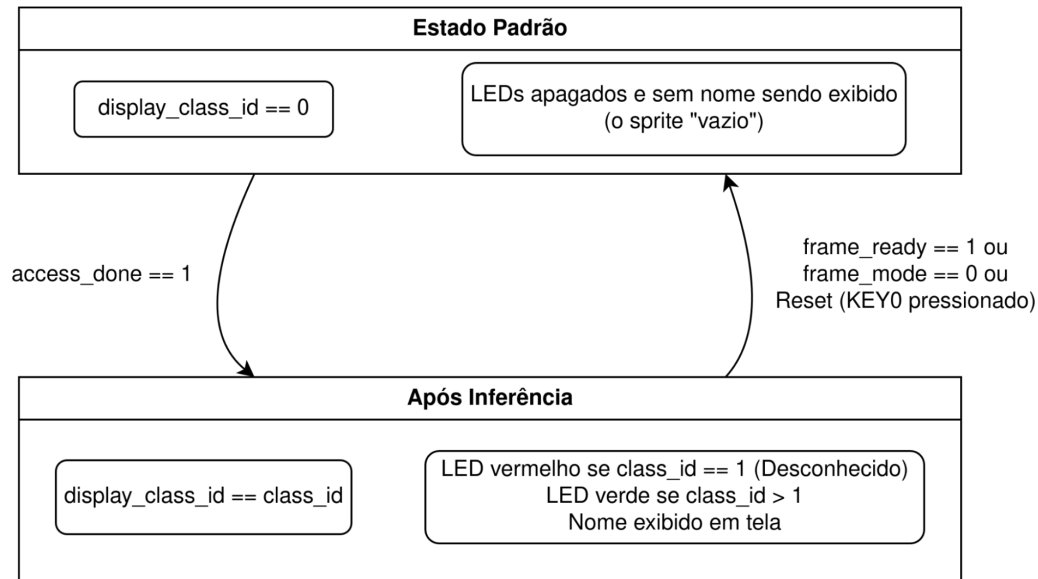


Figura 13 – Diagrama de transição de estados do registrador para exibição dos *sprites*.

5.3 Validação do Artefato Final

Para validar o fluxo final, foi feita uma análise lógica dos sinais utilizando o *software* Quartus. Desejava-se demonstrar que o fluxo da CNN, bem como seu resultado final, eram efetivamente processados de forma correta pela FPGA. Os vídeos referenciados abaixo ilustram esse comportamento.

Demonstração em Vídeo: <<https://link-do-video.com>>

5.4 Problemas Encontrados

Uma das dificuldades encontradas foi a discrepância no volume final de imagens entre os integrantes da equipe. Enquanto a maioria das classes atingiu a meta, as classes referentes a dois integrantes apresentaram volumes inferiores. Essa limitação ocorreu devido à menor duração dos vídeos originais gravados por esses membros, o que ocasionou uma menor quantidade de amostras extraídas. Mesmo com as técnicas de preenchimento

sintético descritas na seção da CNN, o volume final não chegou a uma quantidade de amostras aceitável nestas classes.

Outro problema encontrado no desenvolvimento do trabalho foi a quantização dos valores da rede neural. Apesar dos pesos serem quantizados globalmente em Q1.7, fez-se necessário que cada saída fosse quantizada de maneira otimizada. Logo, a determinação de quantização — como, por exemplo, na saída em Q6.10, devido ao risco de perda de informações cruciais para a acurácia da rede — tornou-se uma questão técnica que ampliou a complexidade do trabalho.

Por fim, a implementação das FSMs de controle representou um desafio para a finalização do projeto, principalmente em relação ao módulo de `argmax`, devido a erros de temporização no Quartus. Para mitigar isso, foi implementada uma FSM dedicada para este módulo da arquitetura em *hardware*, a qual é orquestrada por uma FSM geral (*top-level*), consolidando a integração da rede no sistema embarcado.

6 Conclusão

O presente trabalho demonstrou, de forma concreta e bem-sucedida, a viabilidade do desenvolvimento de um sistema embarcado de reconhecimento facial em tempo real em *hardware* reconfigurável. O projeto integrou conhecimentos de aprendizado de máquina, arquitetura de computadores digitais, comunicação serial e processamento de vídeo em uma única solução de engenharia, atuando de ponta a ponta.

A opção inicial pela rede binária revelou-se inviável diante da necessidade de identificar individualmente cada membro autorizado e exibir seu nome no monitor VGA. A adoção de uma arquitetura *Tiny-CNN* com 19 classes — 18 membros autorizados e uma classe dedicada ao reconhecimento de desconhecidos — resolveu essa limitação. Além disso, permitiu tratar a rejeição de indivíduos não cadastrados como uma classe real e treinável, em vez de recorrer a limiares arbitrários de confiança externos. Essa decisão arquitetural refletiu uma compreensão adequada do problema, pois qualquer indivíduo não cadastrado precisa ser corretamente identificado como tal antes mesmo de se tentar qualquer autorização de acesso.

A etapa de construção do *dataset* e de treinamento do modelo evidenciou a importância do equilíbrio e da diversidade dos dados de entrada. As técnicas de aumento de dados (*data augmentation*) foram essenciais para compensar a limitação natural do volume de capturas e aumentar a capacidade de generalização do algoritmo. A classe de desconhecidos recebeu atenção especial, com a integração de conjuntos de dados externos e imagens sintéticas, o que foi fundamental para minimizar falsos positivos e cobrir a enorme variabilidade do rosto humano fora do escopo de membros cadastrados. O esquema de ponderação diferenciada da função de perda refletiu com precisão a assimetria de custo do problema real: negar acesso a um membro cadastrado é, operacionalmente, mais grave do que classificar um desconhecido como tal.

A quantização dos pesos do modelo treinado para o formato de ponto fixo representou o elo crítico entre o domínio de *software* e o de *hardware*. A estratégia de regularização adotada durante o treinamento cumpriu um papel duplo: além de mitigar o *overfitting*, contribuiu diretamente para a redução do erro de quantização posterior. Isso evidenciou a conexão entre as decisões de treinamento em *software* e as restrições físicas do dispositivo — destacando-se como uma das escolhas mais relevantes do projeto.

A arquitetura de *hardware*, implementada em Verilog, refletiu um domínio consistente sobre os recursos físicos do dispositivo e suas limitações. A máquina de estados finitos orquestrou da forma esperada todos os módulos do sistema, garantindo a correta sequência de operações desde o recebimento da imagem até a saída da inferência.

A adoção do coeficiente de Pearson como métrica principal revelou uma compreensão precisa do que realmente importa para a fidelidade de uma rede neural embarcada: não é necessário que os valores absolutos das ativações intermediárias sejam idênticos entre *software* e *hardware*, mas sim que a proporcionalidade relativa entre eles seja mantida. Os resultados obtidos confirmaram essa premissa com solidez, mantendo uma correlação superior a 0,9997 em todas as camadas analisadas.

A integração do protocolo de comunicação serial como canal de alimentação de dados dinâmicos foi feita de forma unilateral, tendo apenas o FPGA como receptor dessas informações. O protocolo de controle de fluxo estabeleceu uma saída satisfatória para o roteamento dos dados entre os dois modos de operação do sistema: exibição de vídeo contínuo e inferência facial. O subsistema de exibição em monitor completou o ciclo de *feedback* visual de forma eficaz, compondo em tempo real a imagem capturada com os *sprites* de identificação dos membros reconhecidos.

No tocante às dificuldades enfrentadas, a discrepância no volume de imagens entre os integrantes da equipe evidenciou um desafio prático comum em projetos de reconhecimento facial com dados reais: a quantidade e a qualidade dos dados de treinamento por indivíduo impactam diretamente a confiabilidade da classificação para aquela classe específica, mesmo quando técnicas de aumento de dados são empregadas. Esse desequilíbrio, embora não tenha comprometido o funcionamento geral do sistema, serve como lição sobre a importância de padronizar os critérios de coleta desde o início.

Em síntese, o projeto integrou com êxito os quatro artefatos propostos no cronograma — aquisição e treinamento *offline*, arquitetura RTL, *pipeline* de visão e comunicação, e integração final com demonstração prática —, resultando em um sistema coerente e validado quantitativamente. A Fechadura Biométrica Inteligente desenvolvida neste trabalho prova que é possível, com critérios rigorosos de projeto e uma metodologia gradual e incremental, realizar a inferência de redes neurais convolucionais em *hardware* reconfigurável de baixo custo. Isso abre caminho para aplicações de inteligência artificial embarcada em sistemas de segurança autônomos e de baixa latência.

7 Contribuições Individuais

Tabela 14 – Contribuições Individuais

Integrante	Atividades Realizadas
Anna Carolina	• Construção do <i>Dataset</i>
Bruno Alves	• Construção do <i>Dataset</i> • Arquitetura RTL (Implementação e Validação)
Diego Alves	• Construção do <i>Dataset</i> • Arquitetura RTL (Implementação e Validação) • Integração Final (Implementação e Validação) • Elaboração do Relatório
Eduardo Fontes	• Construção do <i>Dataset</i> • Treinamento Offline (Implementação e Validação)
Fábio Alves	• Construção do <i>Dataset</i> • Sistema de Visão (Implementação e Validação) • Integração Final (Implementação e Validação) • Elaboração do Relatório
Felipe Cunha	• Construção do <i>Dataset</i>
Gabriel Wolfovitch	• Construção do <i>Dataset</i> • Arquitetura RTL (Validação) • Integração Final (Implementação e Validação) • Elaboração do Relatório
Hugo Souza	• Construção do <i>Dataset</i>
Igor Giovanni	• Construção do <i>Dataset</i> • Sistema de Comunicação (Implementação e Validação)
João Pedro	• Construção do <i>Dataset</i>
José Henrique	• Construção do <i>Dataset</i>

Continua na próxima página

Continuação da Tabela 14

Integrante	Atividades Principais
Julia Borges	• Construção do <i>Dataset</i> • Arquitetura RTL (Implementação) • Integração Final (Implementação e Validação) • Elaboração do Relatório
Lúcio Teixeira	• Construção do <i>Dataset</i> • Elaboração do Relatório
Naira Beatriz	• Construção do <i>Dataset</i> • Sistema de Visão (Implementação) • Elaboração do Relatório
Rafael Falcão	• Construção do <i>Dataset</i> • Arquitetura RTL (Implementação) • Elaboração do Relatório
Rodrigo Horácio	• Construção do <i>Dataset</i> • Arquitetura RTL (Implementação e Validação) • Elaboração do Relatório
Samuel Ferreira	• Construção do <i>Dataset</i> • Arquitetura RTL (Implementação)
Yuri Damasceno	• Construção do <i>Dataset</i> • Arquitetura RTL (Implementação)
Luiz Carlos	• Sistema de Visão (Implementação)

Referências Bibliográficas

CHOLLET, François. **Keras Documentation**. Disponível em: <<https://keras.io>>. Acesso em: 25 jun. 2026. Nenhuma citação no texto.

TENSORFLOW. **TensorFlow Documentation**. Disponível em: <<https://www.tensorflow.org>>. Acesso em: 25 jun. 2026. Nenhuma citação no texto.

INTEL CORPORATION. **Cyclone IV Device Handbook**. Disponível em: <<https://www.intel.com>>. Acesso em: 25 jun. 2026. Nenhuma citação no texto.

INTEL CORPORATION. **Intel Quartus Prime User Guide**. Disponível em: <<https://www.intel.com/content/www/us/en/docs/programmable>>. Acesso em: 25 jun. 2026. Nenhuma citação no texto.

OPENCV. **Open Source Computer Vision Library**. Disponível em: <<https://opencv.org>>. Acesso em: 25 jun. 2026. Nenhuma citação no texto.

KINGMA, Diederik P.; BA, Jimmy. **Adam: A Method for Stochastic Optimization**. In: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS (ICLR), 2015. Nenhuma citação no texto.

HUANG, Gary B.; RAMESH, Manu; BERG, Tamara; LEARNED-MILLER, Erik. **Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments**. Amherst: University of Massachusetts, 2007. Technical Report UM-CS-2007-049. Nenhuma citação no texto.

OLIVEIRA, Wagner Luiz Alves de. **Projeto de Circuitos Integrados Digitais**. Material da disciplina. Universidade Federal da Bahia. Disponível em: <<https://ava.ufba.br/course/view.php?id=343075>>. Acesso em: 01 abr. 2026. Nenhuma citação no texto.

INTEL CORPORATION. **Intel Quartus Prime Help: Memory Initialization File (MIF)**. Disponível em: <https://www.intel.com/content/www/us/en/programmable/quartushelp/17.0/reference/glossary/def_mif.html>. Acesso em: 20 abr. 2026. Nenhuma citação no texto.

LEARNOPENCV. **2D Convolution Explained: Fundamental Operation in Computer Vision**. YouTube. Disponível em: <<https://www.youtube.com/watch?v=yb2tPt0QVPY>>. Acesso em: 09 abr. 2026. Nenhuma citação no texto.

JHA, Atul Anand. **LFW – People (Face Recognition)**. Kaggle. Disponível em: <<https://www.kaggle.com/datasets/atulanandjha/lfwpeople>>. Acesso em: 09 abr. 2026. Nenhuma citação no texto.

GEEKSFORGEEEKS. **Program to Extract Frames Using OpenCV-Python**. Disponível em: <<https://www.geeksforgeeks.org/python/python-program-extract-frames-using-opencv/>>. Acesso em: 15 abr. 2026. Nenhuma citação no texto.

CENTER FOR RESEARCH IN COMPUTER VISION. UNIVERSITY OF CENTRAL FLORIDA. **Selfie Dataset**. Disponível em: <<https://www.crcv.ucf.edu/data/Selfie/>>. Acesso em: 23 mar. 2026. Nenhuma citação no texto.

TECH TANGENTS. **How VGA Works**. YouTube. Disponível em: <<https://www.youtube.com/watch?v=5exFKr-JJtg>>. Acesso em: 23 mar. 2026. Nenhuma citação no texto.

BLAGOJEVIC, Blagoje. **vga_verilog**. GitHub. Disponível em: <https://github.com/BlagojeBlagojevic/vga_verilog>. Acesso em: 15 maio 2026. Nenhuma citação no texto.